Calculadora Simples

```
print("\033[36mCalculadora Simples\033[0m\n") # Imprime o título da calculad

# Loop para solicitar os dois valores ao usuário
while True:
    try:
        # Solicita o primeiro valor e tenta convertê-lo para inteiro
        valor1 = int(input(" Digite o primeiro valor: ")) # Entrada sem cor
        # Solicita o segundo valor e tenta convertê-lo para inteiro
        valor2 = int(input(" Digite o segundo valor: ")) # Entrada sem cor
        break # Sai do loop se ambos os valores forem válidos
    except ValueError:
        # Exibe mensagem de erro se a entrada não for um número
        print("\033[31mEntrada inválida. Por favor, digite apenas números in

# Loop para solicitar a operação ao usuário
while True:
    print("\n\033[33mEscolha uma operação:\033[0m\n") # Imprime o menu de op
    print("\033[33m  1. Somar\033[0m")
    print("\033[33m  2. Subtrair\033[0m")
    # Solicita a operação e converte para minúsculas para padronização
    operacao = input("\nDigite 'somar' ou 'subtrair' (ou o número correspond

    resultado = None
    operacao_str = ""

    # Verifica qual operação foi escolhida e realiza o cálculo
    if operacao in ['somar', '1']:
        resultado = valor1 + valor2 # Realiza a soma
        operacao_str = "soma"
    elif operacao in ['subtrair', '2']:
        resultado = valor1 - valor2 # Realiza a subtração
        operacao_str = "subtração"
    else:
        # Exibe mensagem de erro se a opção for inválida
        print("\033[31mOpção inválida. Por favor, escolha 'somar' ou 'subtra
        continue # Continua o loop para pedir a operação novamente

    # Se um resultado foi obtido (operação válida), imprime o resultado e sa
    if resultado is not None:
        # Imprime o resultado formatado com cores (magenta e branco)
        print(f"\n\033[35mA {operacao_str} dos valores é: \033[37m{resultado
        break
```

Consultor e Validador CEP

```
import requests
import json
```

```
import time # Importa a biblioteca time para usar a função sleep

def format_cep_input(cep_raw):
    """Remove non-digits and formats CEP for API query."""
    # Remove todos os caracteres não numéricos do CEP
    cep_digits = ''.join(filter(str.isdigit, cep_raw))
    # Formata o CEP se ele tiver 8 dígitos
    if len(cep_digits) == 8:
        return f"{cep_digits[:5]}-{cep_digits[5:]}"
    # Retorna o CEP como está se não tiver 8 dígitos (a API tratará como inv
    return cep_digits

def query_cep_api(cep_formatted):
    """Queries the ViaCEP API and returns the JSON response."""
    url = f"https://viacep.com.br/ws/{cep_formatted}/json/"
    try:
        # Faz a requisição HTTP para a API do ViaCEP
        response = requests.get(url)
        # Verifica se houve erros HTTP (ex: 404, 500)
        response.raise_for_status()
        # Converte a resposta JSON para um dicionário Python
        data = response.json()
        return data
    except requests.exceptions.RequestException as e:
        # Exibe mensagem de erro em caso de falha na conexão ou requisição
        print(f"\033[31mErro ao conectar à API do ViaCEP: {e}\033[0m")
        return None

def display_cep_info(data):
    """Formats and displays the geological search results."""
    # Verifica se os dados foram recebidos e se não há erro na resposta da A
    if data and 'erro' not in data:
        print("\n\033[32mCEP Encontrado:\033[0m")
        print(f"  \033[37mCEP: {data.get('cep')}\033[0m")
        print(f"  \033[37mLogradouro: {data.get('logradouro')}\033[0m")
        print(f"  \033[37mComplemento: {data.get('complemento')}\033[0m")
        print(f"  \033[37mBairro: {data.get('bairro')}\033[0m")
        # Concatena localidade e UF se a UF existir
        print(f"  \033[37mLocalidade: {data.get('localidade')}" + (f" - {dat
        print(f"  \033[37mIBGE: {data.get('ibge')}\033[0m")
        print("\033[32mStatus: CEP válido e encontrado.\033[0m")
    # Se os dados contiverem um erro da API (CEP não encontrado)
    elif data and 'erro' in data:
        print("\n\033[31mStatus: CEP não encontrado ou inválido.\033[0m")
    # Se não foi possível obter informações (erro de conexão, por exemplo)
    else:
        print("\n\033[31mNão foi possível obter informações para o CEP.\033[

# --- Programa Principal ---
print("\033[36m\nBem-vindo ao Validador e Consultor de CEP!\033[0m")

# Variável de controle para o loop principal
should_continue_program = True

# Loop principal para o menu de opções
```

```
while should_continue_program:
    print("\n\033[33mEscolha uma opção:\033[0m\n")
    print("\033[33m  1. Validar CEP\033[0m")
    print("\033[33m  2. Consultar CEP\033[0m")
    print("\033[33m  3. Sair\033[0m")
    # Solicita a escolha do usuário e converte para minúsculas
    choice = input("\nDigite a opção de entrada: ").lower()

    # Opção: Validar CEP
    if choice in ['validar', '1']:
        raw_cep = input("Digite o CEP (apenas números ou com hífen): ")
        # Formata o CEP para a consulta na API
        formatted_cep = format_cep_input(raw_cep)
        # Consulta a API do ViaCEP
        cep_data = query_cep_api(formatted_cep)
        # Verifica se o CEP é válido com base na resposta da API
        if cep_data and 'erro' not in cep_data:
            print(f"\n\033[32mO CEP {formatted_cep} é válido.\033[0m")
        elif cep_data and 'erro' in cep_data:
            print(f"\n\033[31mO CEP {raw_cep} é inválido ou não encontrado.\
        else:
            print("\n\033[31mNão foi possível validar o CEP devido a um erro

    # Opção: Consultar CEP
    elif choice in ['consultar', '2']:
        raw_cep = input("Digite o CEP (apenas números ou com hífen): ")
        # Formata o CEP para a consulta na API
        formatted_cep = format_cep_input(raw_cep)
        # Consulta a API do ViaCEP
        cep_data = query_cep_api(formatted_cep)
        # Exibe as informações do CEP
        display_cep_info(cep_data)

    # Opção: Sair do programa
    elif choice in ['sair', '3']:
        print("\033[32m\nObrigado por usar o Validador e Consultor de CEP!\0
        time.sleep(2) # Pausa por 2 segundos
        should_continue_program = False # Define para sair do loop principal
        continue # Continua para o final do loop while e sai

    # Opção inválida
    else:
        print("\033[31mOpção inválida. Por favor, escolha 'validar', 'consul

    # Se o programa ainda deve continuar (não foi escolhida a opção 'Sair')
    if should_continue_program:
        while True:
            print("\n\033[33mDeseja continuar ou sair?\033[0m\n")
            print("\033[33m  1. Sim (Voltar ao menu principal)\033[0m")
            print("\033[33m  2. Não (Sair do programa)\033[0m")
            continue_choice = input("\nDigite '1' para continuar ou '2' para

            if continue_choice in ['1', 'sim']:
                break # Sai do loop de continuação e volta para o menu princ
            elif continue_choice in ['2', 'não', 'nao']:
```

```
            print("\033[32m\nObrigado por usar o Validador e Consultor d
            time.sleep(2) # Pausa por 2 segundos
            should_continue_program = False # Define para sair do loop p
            break # Sai do loop de continuação
        else:
            print("\033[31mOpção inválida. Por favor, digite '1' para co
```

Adicionar aspas

CONSULTAS DE CEP